

[S.A.P.E CSS]
(SAD) Software Architecture Document
Versión 1.0

Identificación de Documento

Identificación	DAS
Proyecto	S.A.P.E CSS
Versión	1.2

Documento mantenido por	INDIE COMPANY
Fecha de ultima revisión	14-09-2026
Fecha de próxima revisión	22-09-2026

Documento aprobado por	Colegio San Sebastián de Melipilla
Fecha de última aprobación	14-09-2026

Historia de Revisiones

Fecha	Versión	Descripción	Autor
15-09-2026	1.1	Desarrollo hasta el 2.3	José Valenzuela
22-09-2026	1.2	Desarrollo hasta 3.3	José Valenzuela
24-09-2026	1.3	Desarrollo del documento (texto ,img diagramas)	Felipe Catalan

Tabla de Contenidos

1 Introducción	2
1.1 Contexto del Problema y Propósito	3
1.2 Ámbito	3
1.3 Definiciones, acrónimos y abreviaciones	3
1.4 Resumen ejecutivo	4
1.5 Representación	4
Metas y Restricciones de la Arquitectura	4
1.6 Metas de la arquitectura	5
1.7 Restricciones de la Arquitectura	5
1.8 Otros antecedentes y consideraciones	5
2 Vista de Casos de Uso y Escenarios de Calidad	6
2.1 Modelo de Casos de Uso	7
2.2 Especificación de Casos de Uso Relevantes	7
2.3 Especificación de los Escenarios de Calidad Relevantes	8
Vista Lógica	9
Vista de Procesos	11
Vista de Despliegue	12
Vista Física	13
Vista Escenarios 4+1	14
3 Decisiones de Diseño y Selección de Alternativas	15
4 Análisis de Reutilización	17

1 Introducción

1.1 Contexto del Problema y Propósito

El propósito de este documento es tener de manera clara la idea del proyecto, en que consiste, su finalidad y sus diferentes funciones, para así luego poder consultar en caso que se presenten dudas durante el futuro desarrollo de este. Surge para resolver la gestión reactiva de la información escolar en el Colegio San Sebastián de Melipilla, transformando datos históricos estáticos en alertas tempranas procesables.

1.2 Ámbito

El sistema comprende una plataforma web institucional de uso interno con arquitectura de monolito modular: backend en Laravel 12, interfaz SPA en React vía Inertia.js y base de datos PostgreSQL 16, todo orquestado con Docker tanto en desarrollo como en producción en VPS. Está enfocada en el equipo docente y directivo con 3 roles internos: Administrador Director/UTP, Profesor Jefe y Profesor de Asignatura. El Apoderado es receptor externo sin login. Integra ingesta masiva de datos ETL réplica SIGE, analítica con reglas Decreto 67 y asistencia pedagógica mediante IA con tokenización previa. Quedan fuera del MVP el canal SOS y el modelo predictivo con scikit-learn, requisitos futuros

1.3 Definiciones, acrónimos y abreviaciones

ACRÓNIMO	DESCRIPCIÓN
SAD	Documento de arquitectura de software
ETL	Extract, Transform, Load (Extracción, Transformación y Carga de datos).
SIGE	Sistema de Información General de Estudiantes
MVP	Minimum Viable Product: ETL, reglas Decreto 67, IA como borrador y wa.me; sin SOS ni ML predictivo.
RBAC	Control de acceso basado en roles
WA.ME	Click-to-Chat público de WhatsApp; el sistema solo genera el enlace, no envía ni lee mensajes.
HUF / CU	Historia de Usuario Funcional (Jira INDG) / Caso de Uso (DAS).
QS	Escenario de calidad: atributo medible (QS1 tokenización, QS2 rollback).
PG	PostgreSQL 16, base de datos relacional del sistema.
CI/CD	Integración y despliegue continuos (GitHub Actions).
UTP	Unidad Técnico-Pedagógica (rol Administrador junto al Director).

1.4 Resumen ejecutivo

El presente documento define y documenta la arquitectura de software para la Plataforma Web de Analítica Académica, Alerta Temprana y Comunicación Automatizada destinada al Colegio San Sebastián de Melipilla. La solución está diseñada como una aplicación web de página única (SPA) de uso interno institucional bajo una arquitectura Open Source de Laravel, React y PostgreSQL orientada a garantizar un costo cero de licenciamiento comercial para el establecimiento educacional.

El núcleo arquitectónico está diseñado para procesar cargas masivas de datos académicos (ETL) y operar como un motor de diagnóstico que genera indicadores de riesgo escolar fundamentados en el Decreto 67. Para cumplir con la normativa de privacidad y protección de menores (Leyes 19.628 y 21.719), la arquitectura incorpora minimización y tokenización de datos previas a cualquier interacción con la API de Inteligencia Artificial (Google Gemini). Todo el sistema se orquesta con contenedores Docker para asegurar la portabilidad y un despliegue reproducible y sin contratiempos dentro del plazo de construcción de 10 semanas.

1.5 Representación

La arquitectura del sistema del proyecto S.A.P.E CSS está representada siguiendo el enfoque del framework 4+1 y las recomendaciones del proceso unificado. Las vistas incluidas en esta versión del documento son:

- **Vista de Casos de Uso y Escenarios de Calidad:** Describe los casos de uso más significativos, presenta los actores y una descripción de sus casos de uso asociados. De igual forma describe los escenarios de calidad más relevantes para la arquitectura.
- **Vista de Metas y Restricciones:** Describe restricciones tecnológicas, normativas, estándares, etc., los cuales influyen sobre las decisiones arquitectónicas, del producto y del proceso de desarrollo.
- **Vista Lógica:** Describe la estructura estática y modular del sistema. Detalla la separación de responsabilidades mediante la división entre la capa de presentación (Frontend en React), la lógica de negocio y controladores (Backend en Laravel) y el acceso a datos, definiendo los componentes que orquestan el motor ETL y los asistentes de Inteligencia Artificial.
- **Vista de Procesos:** ilustra el comportamiento dinámico del sistema y el flujo de control. Se centra en la concurrencia y los procesos transaccionales críticos, tales como el procesamiento asíncrono de las consultas a la API externa de IA y el mecanismo de cancelación (rollback) durante la ingesta masiva del archivo Excel réplica SIGE en caso de fallos.
- **Vista de Despliegue:** describe la estructura de la plataforma en su entorno de ejecución. Ilustra cómo los artefactos de software como aplicación laravel-app, base de datos PostgreSQL y proxy inverso caddy se mapean y distribuyen como contenedores Docker en el VPS de producción y en las estaciones de desarrollo.
- **Vista Física :** Representa la infraestructura tangible y las redes que conectan el ecosistema. Detalla la interacción entre los dispositivos de los usuarios finales, equipos de escritorio de dirección y teléfonos móviles de los docentes, hacia los servidores que alojan los contenedores, operando bajo protocolos HTTPS para asegurar el tránsito de datos.

Metas y Restricciones de la Arquitectura

A continuación se revisan las metas y restricciones de la arquitectura.

1.6 Metas de la arquitectura

De acuerdo a las reuniones y al análisis de los requerimientos, se listan los principales conductores iniciales de la arquitectura los cuales corresponden a las metas arquitectónicas iniciales:

- **Desempeño:** interacción clara y medible. Para operaciones locales, la carga de dashboards y la confirmación de acciones deberán tener un percentil 95 de hasta 2 segundos bajo la carga de prueba definida por el equipo. Las consultas de IA se procesan de forma asíncrona: la interfaz confirma la recepción en hasta 2 segundos y registra el tiempo total del proveedor, sin garantizar una respuesta bajo un umbral que dependa de un servicio externo como Gemini free-tier.
- **Tolerancia a fallos:** cancelación segura de transacciones durante la importación masiva (ETL) si se detectan anomalías, para evitar corrupción de datos. Medida: 0% de datos parciales o corruptos guardados en PostgreSQL; la interfaz SPA notifica el error en menos de 5 segundos (QS2).
- **Seguridad:** Aplicación estricta de tokenización de identidad antes del consumo de APIs y autorización basada en roles
- **Modificabilidad/Reúso:** separación clara por capas entre la interfaz React vía Inertia.js, el backend Laravel con controladores, acciones y jobs, y el dominio aislado del framework con modelos, servicios y reglas Decreto 67. Medida: cambiar un adaptador externo de proveedor de IA, base de datos o proxy no obliga a reescribir las reglas de negocio.
- **Operatividad:** despliegues uniformes mediante contenedores: el mismo docker compose up levanta el stack en las máquinas de los 3 integrantes y en el VPS de producción: laravel-app + postgres:16, más caddy con SSL automático en producción.

1.7 Restricciones de la Arquitectura

Existen restricciones que han sido levantadas con los stakeholders, las cuales se presentan a continuación:

- **Tiempo de construcción:** El proyecto se encuentra limitado a un periodo de ejecución de 10 semanas
- **Infraestructura:** Todo el sistema debe ser orquestado localmente mediante Docker.
- **Otros componentes de software:** Existe la obligación de desarrollar a costo cero, utilizando lenguajes de código abierto y capas gratuitas de APIs
- **Normativa y privacidad:** el diseño se alinea con la Ley 19.628 y prepara el cumplimiento de la Ley 21.719 (vigencia 01-12-2026). Obliga a minimización, seudonimización y tokenización antes de toda llamada a IA externa, control de acceso por rol, bitácora y datos de prueba 100% sintéticos con Faker; el Apoderado no posee cuenta ni permisos.
- **Alcance del MVP:** el canal SOS con notificaciones push y el modelo predictivo con scikit-learn quedan fuera del MVP como requisitos futuros . La comunicación asistida se realiza solo con enlaces wa.me revisados por el docente, y la analítica con reglas Decreto 67 explicables.

1.8 Otros antecedentes y consideraciones

La arquitectura se apoya en mecanismos concretos de Laravel 12: contenedor de servicios e inyección de dependencias para la modularización con Services, Actions y Jobs, además de Eloquent

ORM sobre PostgreSQL 16, Scheduler y Queues para el trabajo asíncrono de consultas Gemini y ETL pesado, e Inertia.js como puente monolítico entre controladores y componentes React. La seguridad se implementa con Spatie Permission (3 roles internos), middleware de rol y permiso, Políticas por objeto y cierre de sesión por inactividad.

Supuestos verificables: desarrollo y demos operan únicamente con datos sintéticos ficticios con Faker vía factories/seeder en formato réplica SIGE; los servicios externos se usan en tier gratuito con cuotas medidas: Gemini con caché, reintento y recomendaciones de respaldo, y wa.me público sin verificación; el Apoderado es stakeholder externo sin credenciales y solo recibe el enlace generado.

2 Vista de Casos de Uso y Escenarios de Calidad

Esta sección describe en detalle el conjunto de escenarios funcionales y no funcionales que obtuvieron la mayor prioridad en el análisis. Para esto se presenta y describe el diagrama de casos de uso y los casos de uso prioritarios, así como los escenarios en que uno o más atributos de calidad se ven involucrados de manera significativa.

2.1 Modelo de Casos de Uso

El modelo de casos de uso puede ser encontrado en el documento "Casos de Uso".

2.2 Especificación de Casos de Uso Relevantes

Los casos de uso considerados los más relevantes para el desarrollo de la arquitectura fueron determinados. Los criterios usados para dicha determinación fueron:

- Su implementación implica varios nodos de la vista de despliegue.
- Su implementación es de alto riesgo.
- Incluye muchos conceptos y relaciones del dominio.
- Incluye posibles escenarios críticos de calidad.

A continuación se listan los casos de uso relevantes, los cuales pueden ser encontrados con su especificación detallada en el documento "Casos de Uso".

Código	Nombre	Actores	Prioridad
CU-001	Ingesta masiva ETL réplica SIGE con validación y rollback	Administrador (Director / UTP)	Muy Alta
CU-002	Autenticación, autorización por roles y seguridad de sesión	Administrador, Profesor Jefe, Profesor de Asignatura	Muy Alta
CU-003	Detección de riesgo escolar y cuellos de botella Decreto 67	Administrador (Director / UTP), Profesor Jefe	Alta
CU-004	Asistente IA pedagógico por chat con tokenización	Profesor Jefe, Profesor de Asignatura	Alta
CU-005	Redacción asistida de informe de derivación	Profesor Jefe, Profesor de Asignatura	Alta
CU-006	Generación de enlace wa.me y trazabilidad	Profesor Jefe (Apoderado: receptor externo sin login)	Alta
CU-007	Consulta directiva: dashboard, mapa térmico y buscador NL	Administrador (Director / UTP), Profesor Jefe	Media-Alta

2.3 Especificación de los Escenarios de Calidad Relevantes

Después de un análisis en conjunto con los stakeholders, los escenarios de calidad se expresan a continuación. Estos escenarios responden directamente a las metas de "Seguridad" y "Tolerancia a fallos" definidas en la sección 2.1 del documento.

ID: QS1

Nombre: Seguridad y Privacidad en el consumo de API Externa (Tokenización).

Sinopsis: El sistema debe proteger la identidad de los estudiantes al interactuar con servicios externos de Inteligencia Artificial.

Entorno: Profesor utilizando el Asistente Pedagógico bajo condiciones normales de operación.

Cambio en el entorno: El usuario envía una consulta (prompt) que requiere procesar el contexto de un alumno en específico hacia la API de Google Gemini.

Comportamiento esperado: El backend intercepta la petición y aplica minimización y tokenización de datos, reemplazando el RUN y el nombre real por identificadores genéricos antes de enviar la carga útil (payload) a la API. Trazabilidad: CU-004, CU-005.

Medida: El 100% de las peticiones salientes hacia la API de Inteligencia Artificial se ejecutan sin datos directamente identificables, cumpliendo con las Leyes 19.628 y 21.719.

Prioridad Arquitectónica: Muy Alta.

Aplicación: Componente Backend (Laravel) y Servicios Externos.

ID: QS2

Nombre: Tolerancia a fallos: Cancelación segura de transacciones (Rollback) en módulo ETL.

Sinopsis: El sistema debe evitar la corrupción de datos y la carga de información parcial durante la ingesta masiva de archivos.

Entorno: Administrador procesando un archivo Excel réplica SIGE con el historial académico a través de la plataforma web.

Cambio en el entorno: Ocurre un fallo de red con el motor de base de datos o el archivo Excel contiene formato corrupto a mitad del procesamiento.

Comportamiento esperado: El sistema detiene inmediatamente la ejecución y realiza un *rollback* (cancelación) completo de la transacción en la base de datos, asegurando que no se guarden registros incompletos.

Medida: 0% de datos parciales o corruptos guardados en PostgreSQL; la interfaz SPA notifica al usuario del error en un tiempo menor a 5 segundos.

Prioridad Arquitectónica: Alta.

Aplicación: Componente Backend (Laravel) y Base de Datos (PostgreSQL).

ID: QS3

Nombre: Rendimiento en dashboards y acuse asíncrono de IA.

Sinopsis: El sistema debe responder rápido en operaciones locales y acusar de inmediato las solicitudes de IA sin prometer tiempos del proveedor externo.

Entorno: Docente o directivo consultando dashboards y usando el asistente bajo la carga de prueba definida por el equipo.

Cambio en el entorno: Se cargan dashboards con datos consolidados o se envía una consulta al asistente de IA.

Comportamiento esperado: Los dashboards y confirmaciones cargan dentro del umbral; la consulta de IA se encola como Job y la interfaz confirma la recepción de inmediato, registrando el tiempo total del proveedor. Trazabilidad: CU-007, CU-004.

Medida: Percentil 95 de hasta 2 segundos en dashboards y confirmaciones; acuse de recepción de IA en hasta 2 segundos, sin umbral garantizado en la respuesta total del proveedor

Prioridad Arquitectónica: Alta.

Aplicación: Frontend React, laravel-app con Queues y API Gemini.

ID: QS4

Nombre: Autorización por rol y alcance de curso.

Sinopsis: El sistema debe impedir que un usuario vea o modifique datos fuera de su rol y de sus cursos asignados.

Entorno: Profesor Jefe o de Asignatura navegando con sesión válida.

Cambio en el entorno: El usuario intenta abrir un curso, matrícula o informe fuera de su alcance.

Comportamiento esperado: El Middleware y las Políticas deniegan con 403 y registran el intento en bitácora; el Jefe solo opera sus cursos y el Profesor de Asignatura solo sus grupos. Trazabilidad: CU-002.

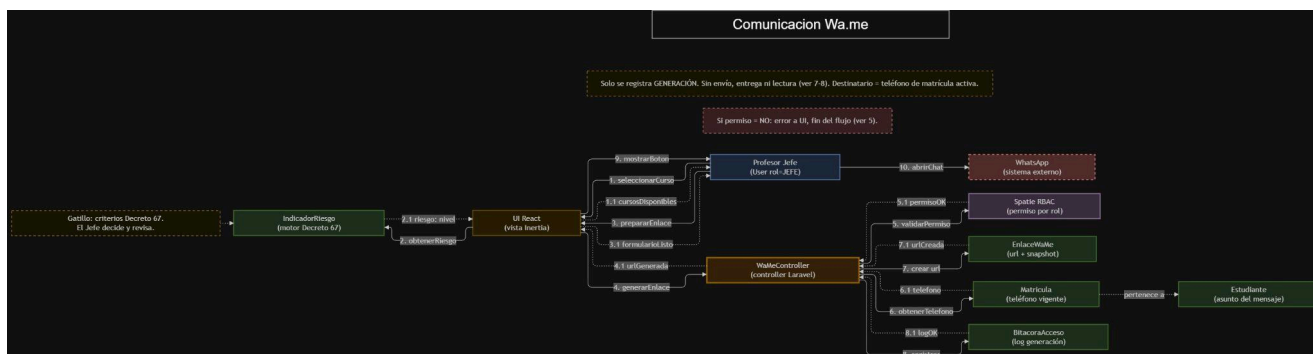
Medida: 100% de accesos fuera de alcance denegados y auditados; 0% de funciones visibles fuera del rol.

Prioridad Arquitectónica: Alta.

Aplicación: laravel-app con Spatie RBAC y base de datos PostgreSQL

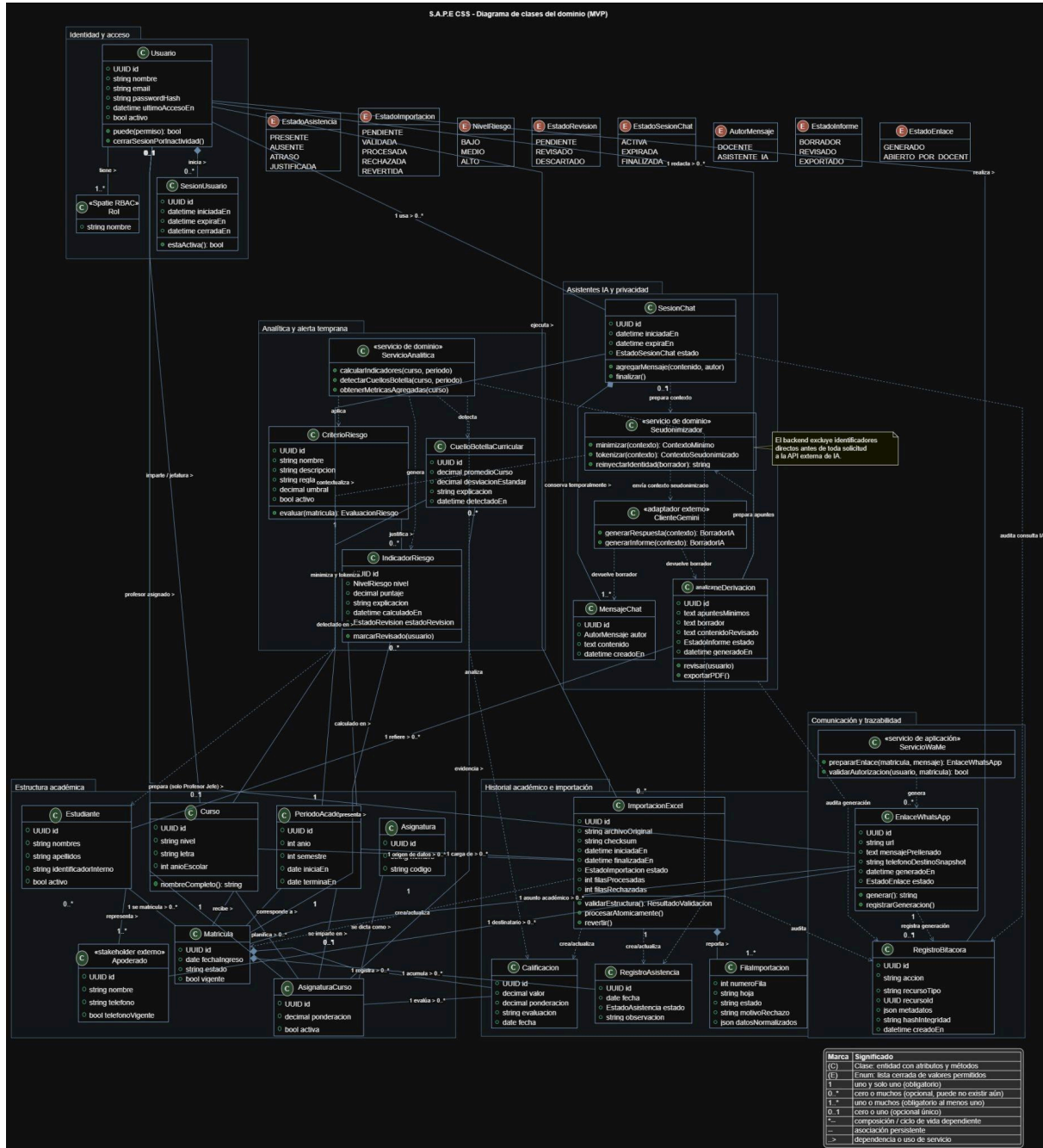
Vista Lógica

Diagrama Comunicación



El diagrama de comunicación modela en diez pasos la generación del enlace de WhatsApp: el Profesor Jefe selecciona su curso, el sistema consulta el riesgo ya calculado por el motor Decreto 67, el Jefe prepara el enlace, el backend construye la URL, se valida el permiso por rol, se obtiene el teléfono vigente de la matrícula, se crea el enlace con su snapshot, se registra la generación en bitácora, se muestra el botón Citar Apoderado y finalmente el Jefe abre el chat en WhatsApp. La idea central es el límite del sistema: los nueve primeros pasos ocurren dentro de la plataforma y son auditable, mientras que el décimo es la puerta de salida. El sistema jamás envía ni confirma mensajes.

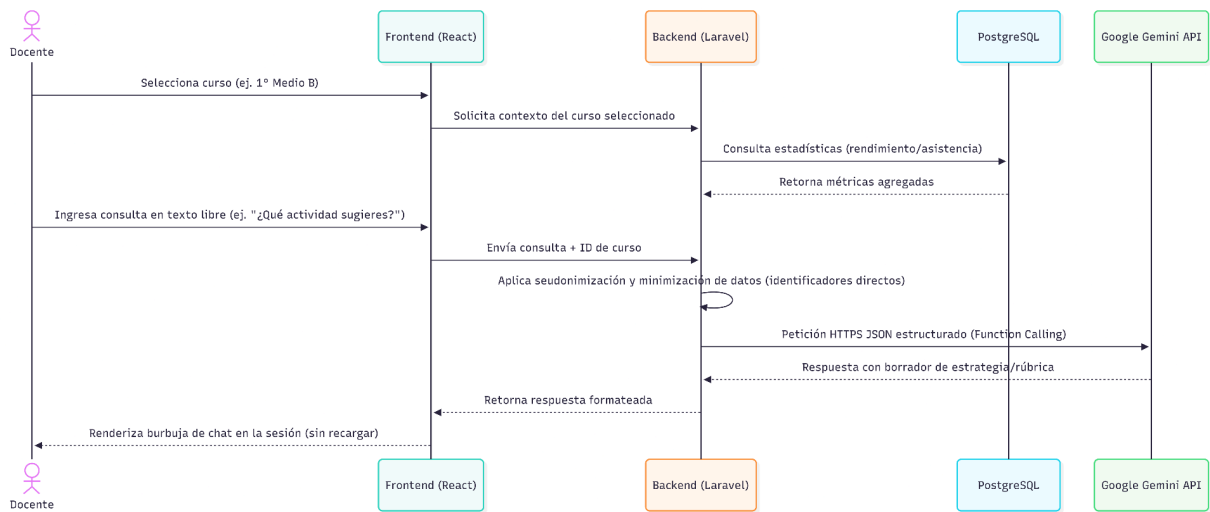
Diagrama de clases



El diagrama de clases representa el dominio del MVP en seis paquetes: Identidad y acceso con Usuario, Rol y SesiónUsuario; Estructura académica con Estudiante, Apoderado, Curso,

PeriodoAcademico, Matricula, Asignatura y AsignaturaCurso; Historial académico e importación con Calificacion, RegistroAsistencia, ImportacionExcel y FilalImportacion; Analítica y alerta temprana con CriterioRiesgo, IndicadorRiesgo, CuelloBotellaCurricular y ServicioAnalitica; Asistentes IA y privacidad con SesionChat, MensajeChat, InformeDerivacion, Seudonimizador y ClienteGemini; y Comunicación y trazabilidad con EnlaceWhatsApp, RegistroBitacora y ServicioWaMe; además de ocho enumeraciones de estados y niveles. Tres reglas se leen directo del diagrama: el Apoderado no es usuario del sistema, la importación es atómica y reversible, y la IA solo trabaja con contexto seudonimizado bajo revisión humana.

diagrama de secuencia



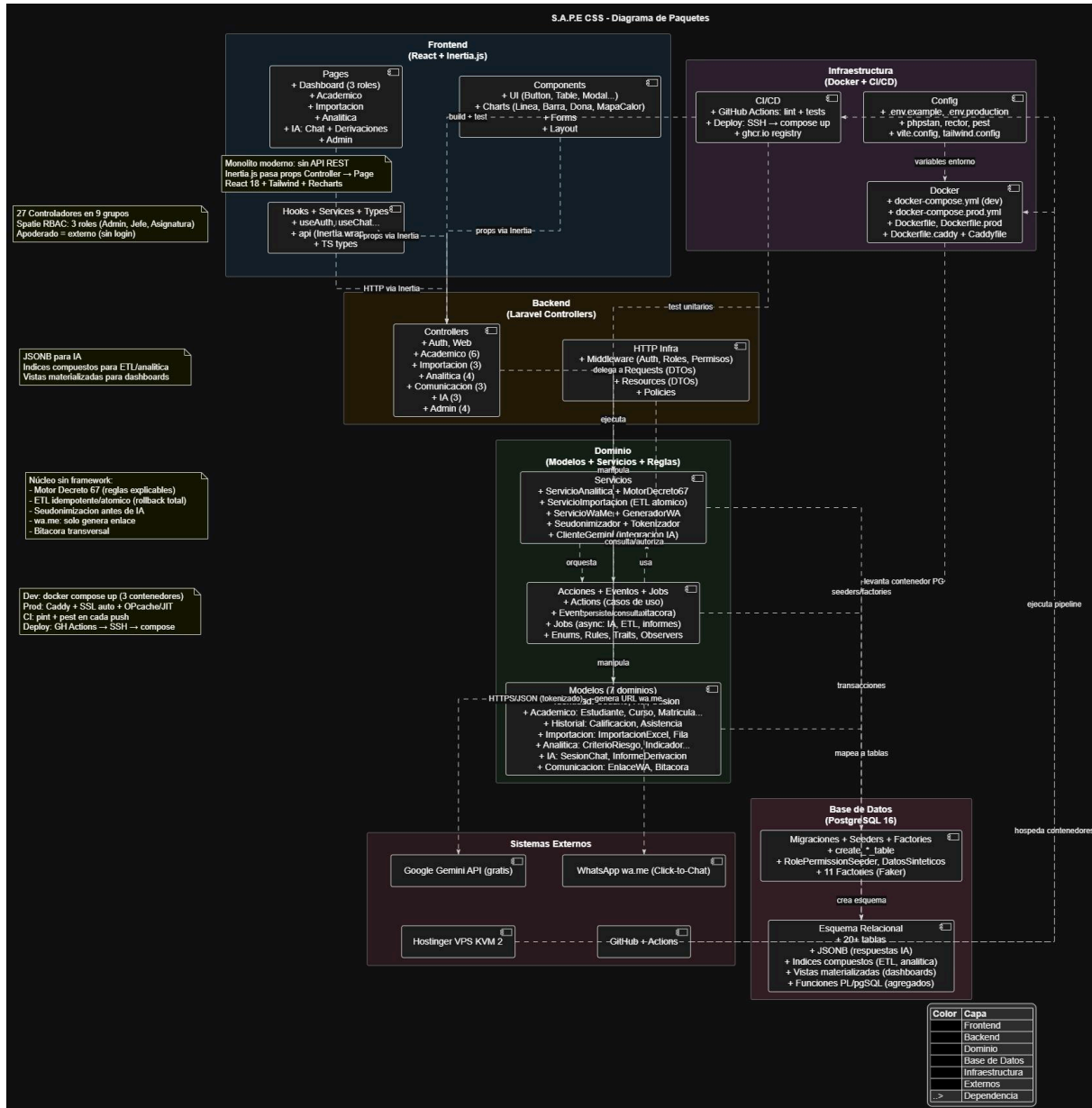
Vista de Procesos

Diagrama de actividad

El diagrama de actividad modela el macro-flujo del sistema agrupando los casos de uso en cuatro fases secuenciales de negocio. Comienza con la Ingesta de Datos, donde el Administrador carga y valida estructuralmente los archivos CSV (UC-01, UC-11). A continuación, en el Análisis y Procesamiento, la directiva evalúa el riesgo escolar mediante dashboards institucionales (UC-04, UC-02) mientras el Profesor Jefe analiza el mapa térmico de su curso (UC-05). Durante la Asistencia Pedagógica, las consultas a la IA (UC-03, UC-07) ejecutan automáticamente la tokenización de identidad (UC-10) para resguardar la privacidad de los estudiantes. Finalmente, el proceso culmina en la Acción y Derivación, donde el sistema genera borradores de informes y enlaces preestructurados de WhatsApp (UC-08, UC-06) que exigen la confirmación explícita del docente o administrador (UC-12) para concretar la alerta temprana.

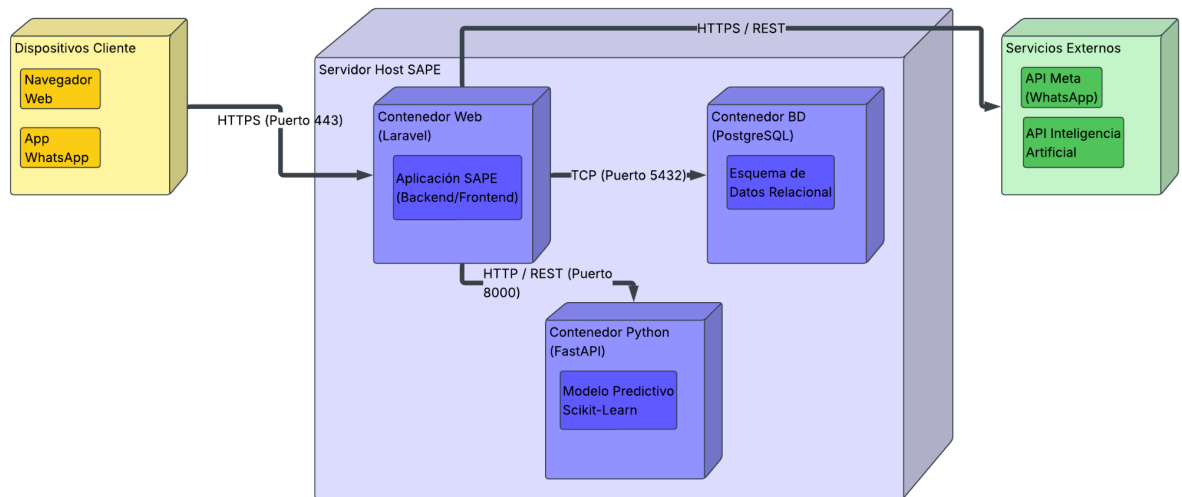
Vista de Despliegue

diagrama de paquetes



El diagrama de paquetes organiza el sistema en seis bloques: Frontend con React e Inertia.js, Backend con los controladores Laravel, Dominio con modelos, servicios y reglas de negocio, Base de Datos PostgreSQL 16, Infraestructura con Docker e integración continua, y Sistemas Externos con Gemini, WhatsApp, el VPS y GitHub. La regla de lectura es única: las dependencias siempre apuntan hacia adentro, hacia el Dominio, que nunca depende de la interfaz, del framework web ni de la infraestructura.

Vista Física



El diagrama de despliegue estructura la plataforma S.A.P.E CSS en tres nodos principales interactuando mediante protocolos seguros. Los Dispositivos Cliente acceden a la interfaz web y a los enlaces preestructurados de WhatsApp exclusivamente vía HTTPS. El Servidor Host SAPE, orquestado con Docker para garantizar portabilidad, aísla el núcleo monolítico (Laravel/React) en un Contenedor Web que se comunica internamente con la Base de Datos (PostgreSQL) por el puerto TCP 5432 y con un microservicio predictivo en Python (FastAPI) vía REST, siendo este último proyectado para futuras versiones del MVP. Finalmente, el servidor consume Servicios Externos en la nube (API Meta y Gemini) mediante peticiones HTTPS, ejecutando siempre una tokenización previa de los datos para garantizar la privacidad de los estudiantes

3 Decisiones de Diseño y Selección de Alternativas

Las decisiones se tomaron bajo tres condicionantes que no se negocian: plazo de construcción de 10 semanas, equipo de 3 personas con primera experiencia en Docker y React, y obligación de costo \$0 en licencias con un solo servidor VPS. En cada decisión se indica la alternativa descartada y su consecuencia.

Decision 1. Monolito Modular frente a microservicios.

un solo VPS KVM 2 y un ETL que exige atomicidad total. Decisión: un único desplegable con laravel-app + postgres:16, más caddy en producción, y con fronteras internas por paquetes: Presentación- Aplicación-Dominio- Persistencia. Se descarta la opción de microservicios con Laravel + FastAPI + frontend separado + gateway porque exige service discovery, observabilidad distribuida y sagas o 2PC para el rollback del ETL, además de más memoria de la disponible, sin beneficio para un solo equipo y un solo despliegue. Consecuencia: despliegue con docker compose up, transacción única en PostgreSQL y depuración local.

Decisión 2. Inertia.js frente a API REST separada con React desacoplado + Sanctum/JWT.

SPA reactiva sin aplicación móvil nativa que consuma una API pública. Decisión: Inertia.js entrega las props del controlador directamente al componente React con sesión segura. Se descarta la API REST separada porque duplica el trabajo: rutas web y api, validación en JS y en FormRequest, tipos TS y Resources, CORS y tokens para el mismo equipo y el mismo plazo. Consecuencia: la mitad del código con la misma reactividad de React + Tailwind + Recharts. Si en el futuro existe una app móvil, se agrega routes/api.php sobre los mismos Actions y Services sin reescribir.

Decision 3. PostgreSQL 16 local contenerizado frente a MySQL o BaaS externo como Supabase o Firebase. Contexto: perfil analítico con consultas agregadas, funciones de ventana y JSONB para respuestas de IA, privacidad de menores (Ley 19.628 y Ley 21.719) y demostración local sin depender de internet. Decisión: postgres:16 en Docker con JSONB, índices compuestos para ETL y analítica, vistas materializadas para dashboards y funciones PL/pgSQL, accedido vía Eloquent. Se descarta MySQL por ser más limitado analíticamente y el BaaS externo por datos de menores en un tercero, pausas por inactividad, latencia, costo variable y riesgo para la demo. Consecuencia: soberanía de los datos y entorno idéntico en desarrollo y producción.

Decisión 4. Motor diagnóstico con reglas Decreto 67 explicables frente a scikit-learn predictivo en el núcleo. El EPT exige indicadores explicables y auditables; no existen históricos consolidados, solo formato réplica SIGE más datos sintéticos Faker, y el plazo es de 10 semanas. Decisión: reglas configurables en Laravel/PostgreSQL, como promedios, asistencia y desvíos por asignatura, donde cada Indicador de Riesgo queda justificado por su Criterio de Riesgo y cada Cuello de Botella Curricular explica datos y umbrales. Se descarta FastAPI + scikit-learn en el MVP porque implica doble stack, doble CI y doble superficie de privacidad

Decisión 5. WhatsApp wa.me frente a WhatsApp Business API. Contexto: costo \$0, sin verificación de negocio y Apoderado como stakeholder externo sin login. Decisión: el ServicioWaMe genera el enlace wa.me Se descarta Business API por costo por mensaje, número dedicado y verificación de negocio, innecesarios para el MVP. Consecuencia: costo \$0

Decisión 6. Gemini free-tier asíncrono con tokenización frente a llamada síncrona directa. Contexto: cuota gratuita con límites, latencia externa no garantizada, RNF de acuse en ≤ 2 segundos y prohibición de enviar identificadores directos. Decisión: la UI acusa recibo en ≤ 2 segundos, el JobConsultaGemini se encola, el Seudonimizador minimiza y tokeniza, el cliente HTTP llama a Gemini con contexto mínimo, se valida el esquema y se guarda un borrador sujeto a revisión

humana, con bitácora, caché y recomendaciones de respaldo. Se descarta la llamada síncrona desde el controlador porque bloquearía la interfaz y expondría timeouts y cuotas al docente. Consecuencia: privacidad por diseño y resiliencia ante límites del tier gratuito.

Todas las decisiones son reversibles por diseño: las integraciones externas viven detrás de adaptadores como ClienteGemini y GeneradorWA, por lo que cambiar de proveedor o extraer un servicio en el futuro no obliga a reescribir el núcleo de dominio.

4 Análisis de Reutilización

Se reutiliza todo lo genérico y se construye solo lo diferencial del dominio escolar chileno: formato réplica SIGE, reglas Decreto 67, tokenización de menores y comunicación wa.me asistida. Así se cumple el costo \$0, el plazo de 10 semanas y la mantenibilidad: cambiar de proveedor de IA, de base de datos o de proxy afecta a adaptadores e infraestructura, no a las reglas de negocio.

Reutilizado: Laravel 12 con autenticación, Eloquent, Scheduler, Queues y validación; starter Breeze React + Inertia como base de autenticación y SPA; Spatie Permission (3 roles internos: Administrador, Profesor Jefe, Profesor de Asignatura; el Apoderado no es rol); React + Tailwind + Recharts/Chart.js para dashboards, mapa térmico y chat; Caddy con SSL automático vía Let's Encrypt; GitHub Actions (lint pint + tests Pest) con ghcr.io; Faker PHP con factories/seeder (datos sintéticos en formato SIGE); imágenes oficiales postgres:16 y node/vite.

Construido: MotorDecreto67 con CalculadorIndicadores y DetectorCuellosBotella, con reglas explicables y auditables; ValidadorEstructura y ProcesadorAtomico con ImportacionExcel::revertir(), con ETL idempotente por hash, reporte estilo SIGE y rollback total; Seudonimizador, Tokenizador y ContextoMinimo, con minimización previa a toda llamada de IA; ClienteGemini con validación de esquemas y borradores con revisión humana; ServicioWaMe con EnlaceWhatsApp y snapshot, que solo genera el enlace; RegistroBitacora transversal de accesos, IA, ETL y enlaces; catálogo de subsectores y criterios Decreto 67 configurables; seeders de casos borde: retirado, cuello de botella en MAT-01, conceptual en REL-01, eximido en EFI-01.